

Week 8 Write Up for
Master of Science
Information and Communications Technology

Melissa Knapp
University of Denver University College

May 24, 2026

Faculty: Nathan Braun, MA

Director: Cathie Wilson, MS

Dean: Michael J. McGuire, MLS

Table of Contents

Table of Contents	i
Overview	1
Challenges Faced.....	1
JUnit Testing	2
Future Considerations.....	4

Overview

The primary challenge pertains to tracking visitors across various parking facilities and invoicing them monthly according to the duration of their parking. The system encompasses functionality for registering customers and their vehicles, as well as documenting parking locations and time entries. This week, we used Serialization, specifically JSON, to transmit information between the client and server created in the codebase.

This addition helped make the architecture more modular because the client could package parking request data into a structured format and the server could deserialize that data for processing and response generation. Using JSON also clarified how information moves through the system, which made it easier to debug communication issues and verify that customer, vehicle, and parking event details were being transferred accurately between components. Although the implementation required careful attention to object structure and data handling, it provided a practical introduction to how distributed applications exchange information in a reliable and readable way.

Challenges Faced

I found this week to be the most challenging so far. The guideposts are normally my roadmap for what I need to code or update and after completing the `ParkingResponse` and `ParkingRequest` objects I struggled with what I needed to update in the codebase to implement them. I ended up making most of the changes to the `ParkingService` class. With a few additional changes in `Server` and `ServerClient`. In the server class I worked with the `ParkingResponse` and used the `toString` method that was created to print the output of executing the request. I used

http status codes 200 and 400, to represent a processed request and an invalid request respectively.

What I changed in ParkingService is that I changed the return type on handle input from a String to the new ParkingResponse class. This also required changing the Command interfaced and its subclasses, such that the return type of “execute” was also ParkingResponse. And I also changed the handle input method so that it took the string from the input stream, converted it into a ParkingRequest object. Once I did that I used getCommand and getProps to get the information needed to pass along to the performCommand method.

JUnit Testing

JUnit testing for the ParkingResponse and ParkingRequest classes was harder than I thought it would be. I was already familiar with JSON before this class so I thought I would have an easier time of creating the strings needed to pass to the constructor. But the need to escape all the quotation marks in the string made it a lot more difficult to see any syntax errors. And I also got some strange error messages from GSON that I couldn’t figure out. So I ended up switching to the JSON library, then I switched to Jackson, before switching back to GSON. I eventually ended up using a mix of single quotation marks and escaped double quotation marks to get the tests to pass. I think it had to do with the nested JSON object inside of the ParkingRequest, but I’m not 100% sure of that.

Even though the testing process was frustrating, it helped me see how valuable unit tests are when working with serialized data. Writing the tests forced me to examine the exact structure of each request and response object and to think more carefully about what the constructors, getters, and string output were supposed to produce. It also showed me that small

formatting mistakes in JSON can create larger debugging problems than expected, especially when the data includes nested values. Once the tests finally passed, I felt more confident that the `ParkingRequest` and `ParkingResponse` classes were behaving consistently and that the serialization logic was working more reliably than I initially assumed.

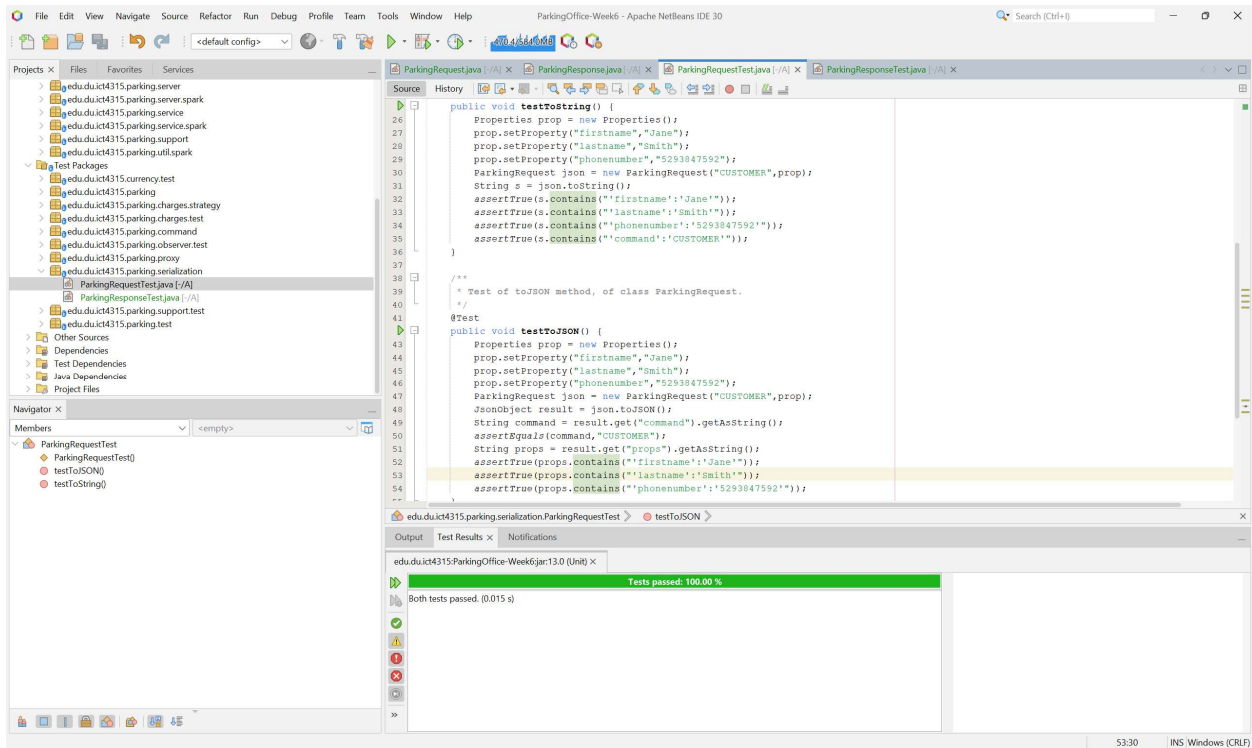


Figure 1. JUnit test results for `ParkingRequestTest`

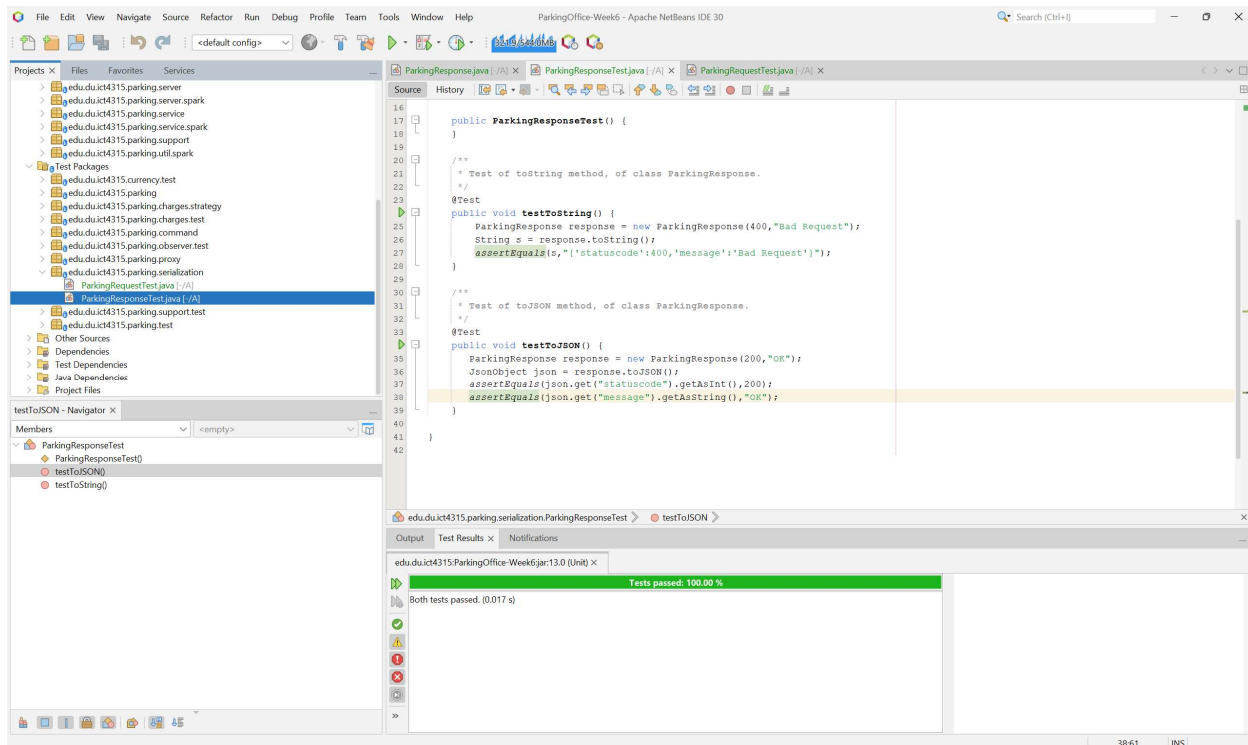


Figure 2. JUnit test results for ParkingResponseTest

Future Considerations

For the future I need to spend more time trying to understand the Client/Server connection. As I mentioned earlier I'm not certain that my implementation is correct. Moving forward, I also need to strengthen my ability to trace requests from the point they are created on the client side to the point they are processed and returned by the server. A deeper understanding of that workflow would help me confirm that the serialized data is not only formatted correctly, but also being interpreted as intended throughout the application.